Table des matières

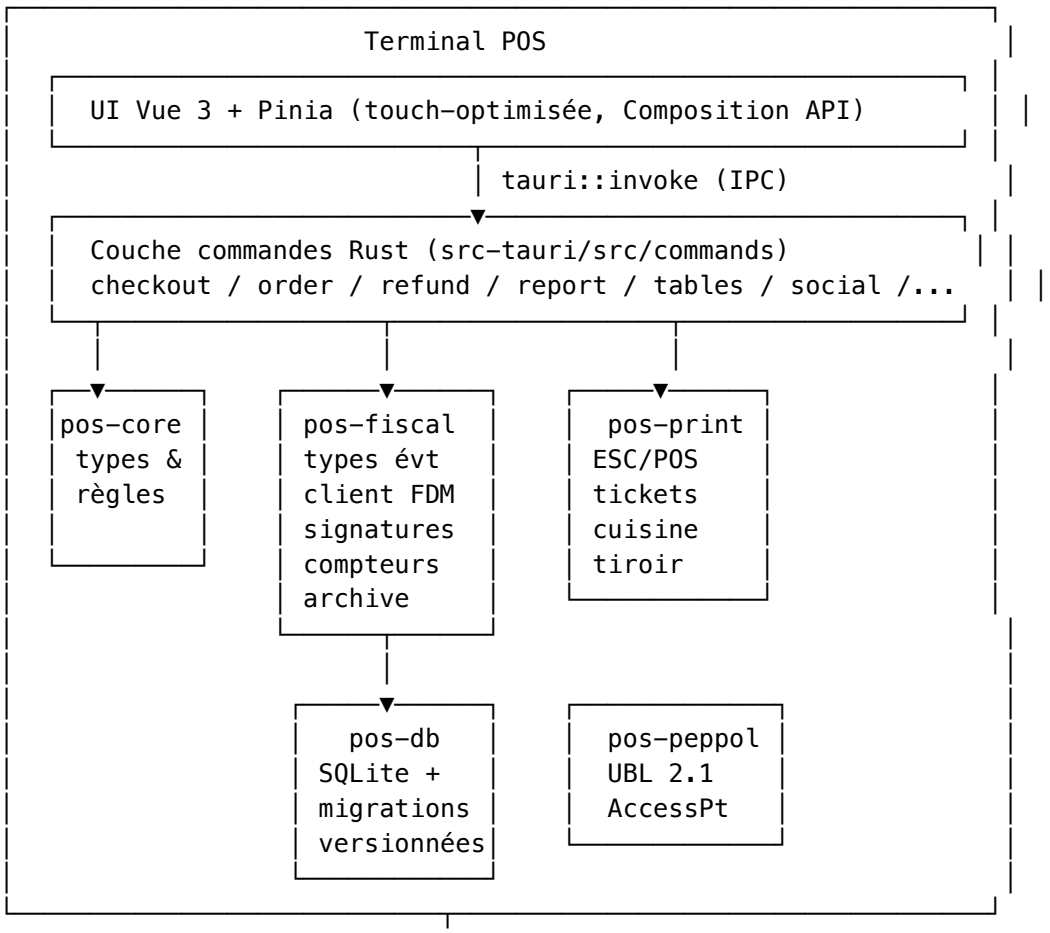
Document de processus technique - moyun-pos SCE 2.0 1
1. Vue d'ensemble du système . . . 1
2. Comportement par événement . . . 2
3. Signature / vérification . . . 2
4. Stratégie de compteurs . . . 3
5. Comportement hors-ligne / déconnexion . . . 3
6. Pipeline d'impression . . . 3
7. Persistance . . . 4
8. Configurations de build . . . 4
9. Carte des sources . . . 4

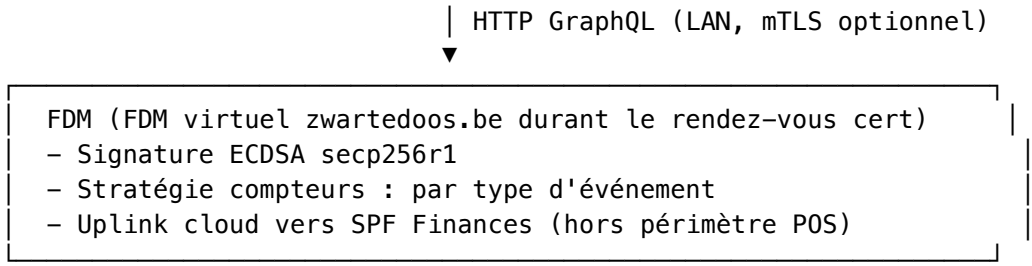
Document de processus technique - moyun-pos SCE 2.0

Distributeur : Bruno Moyse / nuagemagique.dev (Bruxelles, BE) Nom du produit : moyun-pos Version : 1.0.0 (build de soumission cert) Client cible : Tokyo Sushi Bar (Bruxelles, BE) Date de soumission : 2026-05-14 Specs implémentées : POS v2.2 + FDM v1.4 + Addendum 1 + Basic Set v1.7 + Reports v1.3 + Advanced Set v1.5

1. Vue d'ensemble du système

`moyun-pos` est une application desktop Tauri v2 s'exécutant sur Linux x86_64 (Debian 12 minimal + compositeur Cage Wayland en mode kiosk). Elle cible les terminaux POS Penetek A3216 M82 / Advantech USC-360J, se connecte à un FDM certifié SPF Finances par LAN, pilote une imprimante thermique ESC/POS Epson TM-m30III, et persiste chaque événement fiscal dans une base SQLite locale append-only.





2. Comportement par événement

Événement	Déclencheur	Chemin de code POS	Notes
N (Vente normale)	L'opérateur tape "Payer" avec `order_type=dine_in`	`commands/checkout::checkout` → `fdm_client::sign_sale` → archive + close dans une seule tx SQLite	Blocage-sur-déconnexion-FDM imposé
P (Pro forma)	L'opérateur envoie la commande en cuisine	`commands/tables::sign_table_order`	Table ouverte, pas d'incrément du compteur N
T (Formation)	L'opérateur est en mode formation	Comme N mais `is_training=true` dans l'input ; watermark UI	
C (Copie)	Réimpression d'un ticket antérieur	`commands/copy::copy_receipt`	Porte un `fdmRef` vers l'original
I (Facture)	Mode de paiement "Facture" choisi	`commands/checkout` → `sign_invoice`	Si `customer_vat_no` renseigné + Peppol activé → `commands::invoice::issue_b2b_invoic`
F (Financier)	Mouvement caisse, ouverture seule du tiroir	`commands/financial::movement` / `open_drawer_fiscal`	Follow-up ouverture tiroir
S (Social)	Pointage entrée / sortie de l'opérateur	`commands/social::work_in` / `work_out`	INSZ + n° employeur
R (Rapport)	Rapports Z et X	`commands/report::generate_x_report` / `generate_z_report`	Test_x_report + User-Z atomique

Chaque branche :

1. Charge les lignes pertinentes depuis SQLite (les requêtes sont dans `pos-db/src/queries/`).
2. Construit la struct d'input SCE-2.0 (`pos-fiscal::inputs::*`).
3. Calcule la clé d'idempotence (voir §3.4) et le corps JSON canonique (voir §3.3).
4. Appelle la méthode async `FdmClient::sign_*` correspondante.
5. Sur succès : ajoute à `fiscal_events` + ferme/marque la commande dans une seule transaction SQLite.
6. Retourne les données de ticket à l'UI ; l'UI déclenche l'impression via `pos-print`.

3. Signature / vérification

3.1 Transport

— Protocole : GraphQL sur HTTP/HTTPS.
— Endpoint : configuré via `pos.json` (`fdm_url`).

— Sondage de statut : `FdmClient::status` est appelé au démarrage de l'app et à chaque entrée de checkout.

3.2 Signature de réponse

— Algorithme : ECDSA secp256r1.
— Implémentation : `pos-fiscal::signature::verify_short_signature` (crate Rust p256).
— La short signature apparaît sur le ticket imprimé (10 caractères hex) ; la signature complète est stockée dans `fiscal_events.fdm_response_json`.

3.3 JSON canonique

— `pos-fiscal::canonical::to_canonical()` sérialise une valeur en JSON canonique style RFC 8785 : clés triées par codepoint Unicode à chaque niveau, pas d'espace insignifiant, nombres normalisés (pas de zéros traînants, `-0` → `0`).
— Utilisé pour chaque corps de requête envoyé au FDM, de sorte qu'un flux de signature reproductible par hash est possible.
— Test de propriété dans `crates/pos-fiscal/src/canonical.rs::tests` (vérification de déterminisme sur 50 itérations).

3.4 Clé d'idempotence

— `pos-fiscal::inputs::IdempotencyKey { pos_id, pos_date_time, terminal_id, event_label, pos_fiscal_ticket_no }`.
— Chaque type d'input implémente `.idempotency_key()`.
— Permet le retry sûr sur les échecs FDM transitoires : une requête dupliquée avec la même clé est reconnue par le FDM comme une re-soumission et renvoie la signature originale.
— Les tests unitaires dans `crates/pos-fiscal/src/inputs.rs` confirment que deux requêtes sémantiquement différentes au même ticket number déclenchent une erreur de réutilisation de compteur avant l'envoi.

4. Stratégie de compteurs

Voir `STRATEGIE_COMPTEURS_FR.md`. Compteurs par type d'événement stockés dans la table `fiscal_counters` ; alloués sous `BEGIN IMMEDIATE` à l'intérieur de la même transaction SQLite qui ajoute à `fiscal_events`, de sorte que :

— Les compteurs sont strictement monotones par type d'événement.
— Un appel FDM crashé rollback toute la transaction ; le compteur est libéré - pas de trou.
— Au retry, le même `(event_type, pos_fiscal_ticket_no)` est présenté au FDM, qui honore l'idempotence.

5. Comportement hors-ligne / déconnexion

— La build cert bloque sur déconnexion du FDM au point d'entrée du checkout. `commands/checkout::check_fdm_` s'exécute avant chaque transaction ; un échec lève `CheckoutError::FdmUnreachable` et l'UI affiche une modale bloquante avec le dernier statut FDM connu.
— Une feature Cargo dev-only `allow-offline` (default-off) contourne la vérification pour les tests locaux ; elle n'est pas intégrée au binaire cert (`cargo build --release --no-default-features --features cert`).
— Les événements F-WAR d'ouverture tiroir sont quand même émis rétroactivement si le tiroir est ouvert pendant une déconnexion ; l'événement reste dans `fiscal_events` avec un `fdm_response_json` null et un marqueur de statut en attente, rejoué à la reconnexion.

6. Pipeline d'impression

— Crate : `crates/pos-print`.

— Transport : `pos-print::transport` supporte TCP, USB et série. Le déploiement Tokyo Sushi Bar utilise TCP vers la TM-m30III à une IP LAN statique.
— Layout : `pos-print::receipt` formate le ticket fiscal SCE-2.0 selon la spec POS v2.2 §III.3 - en-tête (nom restaurant, adresse, TVA), corps (lignes avec label TVA), table de répartition TVA, bloc de données de contrôle issu de la réponse FDM, QR Code (25×25 modules, Type 2, URL 38 caractères).
— Tickets cuisine : layout séparé (`pos-print::receipt::kitchen`).
— Ouverture tiroir : séquence ESC/POS `0x1B 0x70 0x00 0x19 0xFA` émise dès qu'un paiement `cash` clôture une vente.

7. Persistance

— Moteur : SQLite (DB en un seul fichier dans `~/.local/share/moyun-pos/pos.db` en dev, `/var/lib/moyun-pos/pos.db` en prod).
— Mode : WAL (`PRAGMA journal_mode=WAL`).
— Migrations : versionnées via `pos-db::migrate::run_migrations`. La table `_migrations_applied` suit `(version, name, applied_at)`. Le chemin de rétro-compatibilité pour la migration one-shot d'origine est dans `migrate::backfill_legacy_db`.
— Log fiscal append-only : table `fiscal_events`. Le schéma impose la contrainte CHECK sur `event_type`, et le code applicatif n'émet jamais `DELETE` ni `UPDATE` à son encontre.
— Sauvegarde : snapshot nocturne via l'API SQLite Online Backup (`commands::report::backup_database`) avant chaque rapport Z.

8. Configurations de build

Build	Features Cargo	Produit
Dev	(par défaut) `peppol` + intégrations tierces	App complète avec toutes les intégrations
Cert	`--no-default-features --features cert`	Binaire sans intégrations tierces ; `cert` implique `peppol`
Dev-offline	`--features allow-offline`	Comme dev, plus bypass FDM-déconnexion pour tests E2E

Vérification `nm` sur le binaire cert :

```
$ nm target/release/moyun-pos | grep -c pos_hubrise
0
```

9. Carte des sources

Préoccupation	Crate / fichier
Types d'événements fiscaux & structs d'input	`crates/pos-fiscal/src/inputs.rs`
Client GraphQL FDM	`crates/pos-fiscal/src/fdm_client.rs`
JSON canonique	`crates/pos-fiscal/src/canonical.rs`
Vérification de signature	`crates/pos-fiscal/src/signature.rs`
Compteurs fiscaux	`crates/pos-fiscal/src/counter.rs`
Archivage tickets	`crates/pos-fiscal/src/archive.rs`
Schéma base de données	`crates/pos-db/src/schema.rs`
Runner de migrations	`crates/pos-db/src/migrate.rs`
CRUD commandes	`crates/pos-db/src/queries/orders.rs`
Helpers requêtes rapport Z	`crates/pos-db/src/queries/reports.rs`
Flux checkout	`src-tauri/src/commands/checkout.rs`
Tables / split / merge	`src-tauri/src/commands/tables.rs`
Remboursement + note de crédit	`src-tauri/src/commands/refund.rs`

Préoccupation	Crate / fichier
Rapports (X/Z)	`src-tauri/src/commands/report.rs`
Social (événements S)	`src-tauri/src/commands/social.rs`
Financier (événements F)	`src-tauri/src/commands/financial.rs`
Copie (événements C)	`src-tauri/src/commands/copy.rs`
Sérialiseur UBL Peppol	`crates/pos-peppol/src/ubl.rs`
AccessPoint Peppol	`crates/pos-peppol/src/access_point.rs`
IPC Peppol	`src-tauri/src/commands/invoice.rs`
Impression ESC/POS	`crates/pos-print/src/escpos.rs`
Layout ticket	`crates/pos-print/src/receipt.rs`
Transport imprimante	`crates/pos-print/src/transport.rs`